

```

# BLOB Principle - campagne thermalisee autonome
# N utilise que des apostrophes simples pour eviter conversion mobile

import numpy as np
import time
from collections import Counter, defaultdict

KAPPA_0 = 8
BETA = 0.30
TAU = 0.50

N_LIST = [3000, 5000, 10000]
N_SEEDS = 3
BURN_FACTOR = 5
PROD_FACTOR = 5

class BlobThermal:
    def __init__(self, N, seed):
        self.N = N
        self.rng = np.random.default_rng(seed)
        self.E_target = int(N * KAPPA_0 / 2)
        self.adj = [set() for _ in range(N)]
        self.edge_list = []
        self._init_graph()
        self.deg = np.array([len(self.adj[i]) for i in range(N)], dtype=float)
        self.n_tri = self._count_triangles()
        self.S_curr = self._compute_S(self.n_tri)

    def _init_graph(self):
        for i in range(self.N):
            u, v = i, (i + 1) % self.N
            if v not in self.adj[u]:
                self.adj[u].add(v)
                self.adj[v].add(u)
                self.edge_list.append((min(u, v), max(u, v)))
        while len(self.edge_list) < self.E_target:
            u = int(self.rng.integers(self.N))
            v = int(self.rng.integers(self.N))
            if u != v and v not in self.adj[u]:
                self.adj[u].add(v)
                self.adj[v].add(u)
                self.edge_list.append((min(u, v), max(u, v)))

```

```

def _count_triangles(self):
    tri = sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list)
    return tri // 3

def _compute_S(self, n_tri_val):
    mu = self.deg.mean()
    D = self.deg.std() / mu if mu > 0 else 0.0
    return np.log(1 + n_tri_val) - BETA * D

def step(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1
    self.deg[v_rm] -= 1
    dt_add = len(self.adj[u_add] & self.adj[v_add])
    self.adj[u_add].add(v_add)
    self.adj[v_add].add(u_add)
    self.deg[u_add] += 1
    self.deg[v_add] += 1
    new_n_tri = self.n_tri - dt_rm + dt_add
    if new_n_tri < 0:
        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False
    new_S = self._compute_S(new_n_tri)
    log_alpha = self.N * (new_S - self.S_curr) / TAU
    if log_alpha >= 0:
        accepted = True
    else:
        accepted = self.rng.random() < np.exp(max(log_alpha, -500))
    if accepted:

```

```

        self.edge_list[idx] = (u_add, v_add)
        self.n_tri = new_n_tri
        self.S_curr = new_S
        return True
    else:
        self.adj[u_add].discard(v_add)
        self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1
        self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm)
        self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1
        self.deg[v_rm] += 1
        return False

def run_seed(N, seed, burn_in, production):
    t_start = time.time()
    print(' [N=' + str(N) + ' seed=' + str(seed) + '] Init...', flush=True)
    sim = BlobThermal(N=N, seed=seed)
    ntri_init = sim.n_tri
    S_init = sim.S_curr
    msg = ' [N=' + str(N) + ' seed=' + str(seed) + '] E=' + str(len(sim.edge_list))
    msg += ' n_tri_init=' + str(ntri_init)
    msg += ' S_init=' + '{:.4f}'.format(S_init)
    msg += ' init_time=' + '{:.1f}'.format(time.time() - t_start) + 's'
    print(msg, flush=True)

    print(' [N=' + str(N) + ' seed=' + str(seed) + '] Burn-in ' + str(burn_in) +
          ' t_burn = time.time()')
    acc_burn = 0
    checkpoints = [burn_in // 4, burn_in // 2, 3 * burn_in // 4]
    for i in range(burn_in):
        if sim.step():
            acc_burn += 1
        if i + 1 in checkpoints:
            acc_cur = acc_burn / (i + 1)
            msg = ' burn ' + str(i + 1) + '/' + str(burn_in)
            msg += ' acc=' + '{:.3f}'.format(acc_cur)
            msg += ' n_tri=' + str(sim.n_tri)
            msg += ' S_per_N=' + '{:.4e}'.format(sim.S_curr / N)
            msg += ' t=' + '{:.0f}'.format(time.time() - t_burn) + 's'
            print(msg, flush=True)
    acc_rate_burn = acc_burn / burn_in
    ntri_after_burn = sim.n_tri
    msg = ' [N=' + str(N) + ' seed=' + str(seed) + '] Burn done. acc='
    msg += '{:.3f}'.format(acc_rate_burn) + ' n_tri=' + str(ntri_after_burn)

```

```

msg += ' time=' + '{:.0f}'.format(time.time() - t_burn) + 's'
print(msg, flush=True)

print(' [N=' + str(N) + ' seed=' + str(seed) + '] Production ' + str(production)
acc_prod = 0
S_samples = []
ntri_samples = []
sample_stride = max(1, production // 200)
for i in range(production):
    if sim.step():
        acc_prod += 1
    if i % sample_stride == 0:
        S_samples.append(sim.S_curr)
        ntri_samples.append(sim.n_tri)
acc_rate_prod = acc_prod / production
S_samples = np.array(S_samples)
ntri_samples = np.array(ntri_samples)

mid = len(S_samples) // 2
S_first = S_samples[:mid].mean()
S_second = S_samples[mid:].mean()
drift_rel = abs(S_second - S_first) / abs(S_first) if S_first != 0 else 0.0
ntri_final = sim.n_tri
total_time = time.time() - t_start

if acc_rate_prod == 0:
    verdict = 'FROZEN'
elif acc_rate_prod > 0.95:
    verdict = 'NOT_THERMALIZED'
elif drift_rel > 0.10:
    verdict = 'NOT_CONVERGED'
elif acc_rate_prod > 0.80:
    verdict = 'WARM_BUT_CONVERGED'
elif 0.20 <= acc_rate_prod <= 0.60:
    verdict = 'V93_COMPLIANT'
else:
    verdict = 'ACCEPTABLE'

msg = ' [N=' + str(N) + ' seed=' + str(seed) + '] Prod done. acc='
msg += '{:.3f}'.format(acc_rate_prod)
msg += ' n_tri=' + str(ntri_final)
msg += ' drift=' + '{:.3%}'.format(drift_rel)
msg += ' verdict=' + verdict
msg += ' total=' + '{:.0f}'.format(total_time) + 's'
print(msg, flush=True)

return {

```

```

        'N': N, 'seed': seed,
        'burn_in': burn_in, 'production': production,
        'acc_burn': acc_rate_burn, 'acc_prod': acc_rate_prod,
        'ntri_init': int(ntri_init),
        'ntri_after_burn': int(ntri_after_burn),
        'ntri_final': int(ntri_final),
        'ntri_mean_prod': float(ntri_samples.mean()),
        'ntri_std_prod': float(ntri_samples.std()),
        'S_first_half': float(S_first),
        'S_second_half': float(S_second),
        'S_per_N_mean': float(S_samples.mean() / N),
        'S_per_N_std': float(S_samples.std() / N),
        'drift_rel': float(drift_rel),
        'verdict': verdict,
        'total_time_sec': float(total_time),
    }

print('=' * 70)
print('  BLOB CAMPAIGN - Autonomous Colab')
print('=' * 70)
print('  kappa_0=' + str(KAPPA_0) + ', beta=' + str(BETA) + ', tau=' + str(TAU))
print('  N_list  = ' + str(N_LIST))
print('  Seeds   = ' + str(N_SEEDS) + ' per N')
print('  burn_in = ' + str(BURN_FACTOR) + ' x N')
print('  prod    = ' + str(PROD_FACTOR) + ' x N')
print('  Total   = ' + str(len(N_LIST) * N_SEEDS) + ' simulations')
print('=' * 70)

all_results = []
t_campaign = time.time()

for N in N_LIST:
    burn = BURN_FACTOR * N
    prod = PROD_FACTOR * N
    for seed in range(N_SEEDS):
        print('')
        print('-' * 70)
        print('  Simulation: N=' + str(N) + ' seed=' + str(seed)
              + ' burn=' + str(burn) + ' prod=' + str(prod))
        print('-' * 70)
        r = run_seed(N, seed, burn, prod)
        all_results.append(r)

total_time = time.time() - t_campaign

print('')

```

```

print('=' * 70)
print('  RAPPORT FINAL')
print('=' * 70)
print('  Temps total campagne : ' + '{:.1f}'.format(total_time / 60) + ' min')
print('')
print('  Detail par simulation :')
header = '  ' + 'N'.ljust(7) + 'seed'.ljust(6) + 'acc_burn'.ljust(10)
header += 'acc_prod'.ljust(10) + 'ntri_init'.ljust(12) + 'ntri_fin'.ljust(11)
header += 'drift'.ljust(10) + 'S_per_N'.ljust(14) + 'verdict'
print(header)
print('  ' + '-' * 90)
for r in all_results:
    line = '  ' + str(r['N']).ljust(7) + str(r['seed']).ljust(6)
    line += '{:.3f}'.format(r['acc_burn']).ljust(10)
    line += '{:.3f}'.format(r['acc_prod']).ljust(10)
    line += str(r['ntri_init']).ljust(12)
    line += str(r['ntri_final']).ljust(11)
    line += '{:.3%}'.format(r['drift_rel']).ljust(10)
    line += '{:.4e}'.format(r['S_per_N_mean']).ljust(14)
    line += r['verdict']
    print(line)

by_N = defaultdict(list)
for r in all_results:
    by_N[r['N']].append(r)

print('')
print('  Moyennes par N :')
print('  ' + 'N'.ljust(8) + 'acc_prod'.ljust(12) + 'S_per_N'.ljust(16)
      + 'ntri'.ljust(12) + 'drift'.ljust(12))
print('  ' + '-' * 60)
summary_rows = []
for N in sorted(by_N.keys()):
    rs = by_N[N]
    acc = np.mean([r['acc_prod'] for r in rs])
    SN = np.mean([r['S_per_N_mean'] for r in rs])
    ntri = np.mean([r['ntri_final'] for r in rs])
    drift = np.mean([r['drift_rel'] for r in rs])
    summary_rows.append((N, acc, SN, ntri, drift))
    line = '  ' + str(N).ljust(8)
    line += '{:.3f}'.format(acc).ljust(12)
    line += '{:.4e}'.format(SN).ljust(16)
    line += '{:.1f}'.format(ntri).ljust(12)
    line += '{:.3%}'.format(drift).ljust(12)
    print(line)

if len(summary_rows) >= 2:

```

```

Ns = np.array([row[0] for row in summary_rows])
SNs = np.array([row[2] for row in summary_rows])
log_N = np.log(Ns)
log_SN = np.log(SNs)
slope, intercept = np.polyfit(log_N, log_SN, 1)
ss_res = np.sum((log_SN - (slope * log_N + intercept)) ** 2)
ss_tot = np.sum((log_SN - log_SN.mean()) ** 2)
R2 = 1 - ss_res / ss_tot if ss_tot > 0 else 0
b = -slope
print('')
print('  LOI D ECHELLE :')
print('    S_per_N = ' + '{:.3f}'.format(np.exp(intercept)) + ' x N^(-' + '{:'.format(
print('    R2 = ' + '{:.4f}'.format(R2))
print('    b v93 (thermalise) = 0.870')
print('    b B3 (saturation) = 1.000')
if abs(b - 0.870) < abs(b - 1.000):
    print('    -> Plus proche v93 (ecart ' + '{:.3f}'.format(abs(b - 0.870))
        + ' vs B3 ' + '{:.3f}'.format(abs(b - 1.000)) + ')')
else:
    print('    -> Plus proche B3 (ecart ' + '{:.3f}'.format(abs(b - 1.000))
        + ' vs v93 ' + '{:.3f}'.format(abs(b - 0.870)) + ')')

print('')
print('  DIAGNOSTICS :')
verdicts = [r['verdict'] for r in all_results]
for v, count in Counter(verdicts).most_common():
    print('    ' + v.ljust(25) + ' : ' + str(count) + '/' + str(len(all_results)))

print('')
print('  EVOLUTION DE n_tri (init -> apres burn -> final) :')
for r in all_results:
    delta_burn = r['ntri_after_burn'] - r['ntri_init']
    delta_prod = r['ntri_final'] - r['ntri_after_burn']
    s = '    N=' + str(r['N']) + ' seed=' + str(r['seed']) + ': '
    s += str(r['ntri_init']) + ' -> ' + str(r['ntri_after_burn'])
    s += ' (' + ('+' if delta_burn >= 0 else '') + str(delta_burn) + ')'
    s += ' -> ' + str(r['ntri_final'])
    s += ' (' + ('+' if delta_prod >= 0 else '') + str(delta_prod) + ')'
    print(s)

print('')
print('=' * 70)
print('  FIN DU RAPPORT')
print('=' * 70)

```